Privacy-Preserving Employee Database

Partial Homomorphic Encryption and Searchable Encryption

Alexandre Santos 72970
Miguel Mestre 73018

June 8, 2026

1 Introduction

The objective of this project is to build a secure employee database where the company can store employee information on an untrusted MySQL server without revealing plaintext data. The server is assumed to be honest-but-curious: it correctly stores data and executes queries, but may try to infer information from stored ciphertexts, indexes, query patterns, or result sizes.

The implemented system protects the database using several cryptographic techniques:

- AES encryption for confidential employee attributes.
- HMAC-SHA256 indexes for exact-match searchable encryption.
- Paillier cryptosystem for additive homomorphic salary computations.
- Order-preserving encryption for sorting and comparison over salary and age.
- ECDSA signatures for authenticity of encrypted records.
- HMAC-SHA256 for integrity verification.

The project supports the required operations over encrypted data, including search by employee ID, search by full name, search by department, salary ordering, age ordering, highest salary, oldest employee, salary comparison, salary conversion to USD, department payroll sum, and bonus calculation.

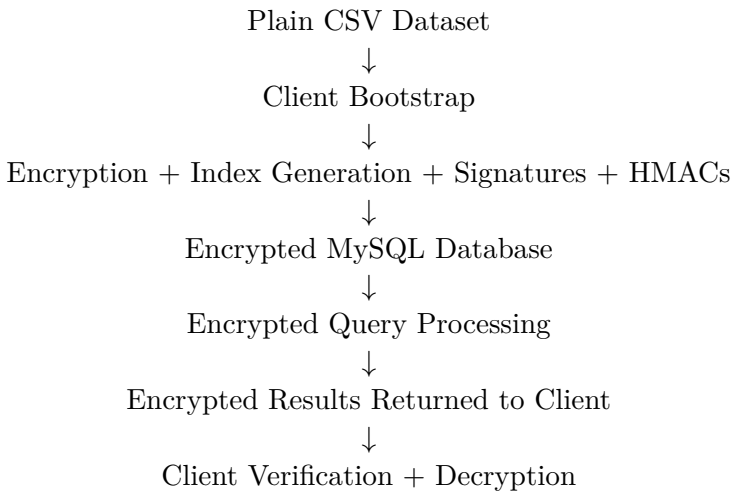
2 System Design Assumptions

2.1 Architecture

The system is divided into two parts:

- **Client side**: trusted component that owns all cryptographic keys, encrypts the dataset, generates indexes, sends encrypted queries, verifies results, and decrypts returned data.
- **Server side**: untrusted MySQL database that stores encrypted records and processes queries directly over encrypted values or searchable indexes.

The general architecture is:



The MySQL server never receives secret keys. It stores only AES ciphertexts, HMAC indexes, Paillier ciphertexts, OPE values, HMAC integrity tags, and ECDSA signatures.

2.2 Implementation Structure

The main project packages are:

```
pa2.client   -> bootstrap, query client, interactive menu, fixed tests
pa2.crypto   -> AES, HMAC, Paillier, OPE, ECDSA, key handling
pa2.model    -> plaintext, encrypted, and decrypted employee models
pa2.server   -> MySQL repository and encrypted query processing
pa2.config   -> MySQL configuration
```

The project includes two client applications:

- `InteractiveClientApp`: terminal menu allowing the user to manually test operations 1 to 11.
- `FixedTestsClientApp`: automatic execution of the 11 required operations using fixed test values.

3 Database Encryption Choices

The plaintext dataset contains employee attributes such as employee ID, full name, age, email, phone numbers, job title, department ID, hire date, employment type, salary, salary band, and bonus eligibility.

Each attribute is protected according to the operations that must be supported.

Attribute	**Protection**	**Reason**
Employee ID	AES + HMAC-SHA256 index	AES protects the value; HMAC index allows exact search.
Full Name	AES + HMAC-SHA256 index	Allows encrypted search by full name.
Department ID	AES + HMAC-SHA256 index	Allows encrypted search by department.
Age	AES + OPE	AES protects the value; OPE allows ordering and oldest employee query.
Salary	AES + Paillier + OPE	AES for retrieval, Paillier for payroll/bonus/conversion, OPE for sorting and comparison.

Attribute	Protection	Reason
Bonus Eligibility	AES + HMAC-SHA256 index	Allows finding bonus-eligible employees.
Other fields	AES	Confidential storage and retrieval.
Whole record	HMAC-SHA256 + ECDSA	Integrity and authenticity verification.

3.1 AES Encryption

AES is used to encrypt readable employee attributes. In the implementation, AES-GCM is used because it provides confidentiality and authenticated encryption. The actual plaintext values are never stored in MySQL.

3.2 HMAC-SHA256 Indexes

For exact-match searches, the client computes:

$$index = HMAC{-}SHA256(key, normalized\_value)$$

The server stores the index and compares it during queries. Since the server does not know the HMAC key, it cannot compute indexes for guessed values without the key. HMAC indexes are used for employee ID, full name, department ID, and bonus eligibility.

3.3 Paillier Homomorphic Encryption

Paillier is used for salary computations. It supports additive homomorphism:

$$Enc(a) \cdot Enc(b) \bmod n^2 = Enc(a + b)$$

and multiplication by a known constant:

$$Enc(a)^k \bmod n^2 = Enc(k \cdot a)$$

This allows the server to compute department payroll sums, salary conversion, and bonus values without decrypting salaries.

3.4 Order-Preserving Encryption

Salary and age require ordering operations. For these fields, the system uses a mutable OPE construction based on the example provided in class. It maintains a plaintext-to-ciphertext ordered map and assigns ciphertexts so that plaintext order is preserved.

If:

$$a < b$$

then:

$$OPE(a) < OPE(b)$$

This allows MySQL to execute `ORDER BY salary_ope` and `ORDER BY age_ope`. The main limitation is that OPE leaks the relative order of salaries and ages.

4 Bootstrap Process

The bootstrap process prepares the encrypted database before any query is executed.

The steps are:

1. Load the plaintext CSV dataset.
2. Generate or load client-side cryptographic keys.
3. Parse each row into an `EmployeePlain` object.
4. Create separate OPE structures for salary and age.
5. Encrypt employee fields with AES.
6. Generate HMAC-SHA256 indexes for searchable fields.
7. Encrypt salaries with Paillier.
8. Generate OPE values for salary and age.
9. Build a canonical representation of each encrypted record.
10. Compute HMAC-SHA256 integrity tag.
11. Generate ECDSA signature.
12. Insert the encrypted record into MySQL.

The client stores its keys locally in:

```
client_keys/client.properties
```

This file must not be uploaded to the server or to the public repository.

The encrypted MySQL table contains only encrypted values and cryptographic metadata. The server never receives the original CSV plaintext values during normal database operation.

5 Supported Operations

The implemented system supports the 11 required operations.

1. **Search by Employee Identification**: the client generates an HMAC token for the employee ID and the server searches `employee_id_index`.
2. **Search by Employee Full Name**: the client generates an HMAC token for the full name and the server searches `full_name_index`.
3. **Employees ordered by Salary**: the server sorts records using `salary_ope`.
4. **Employees by Department**: the client generates an HMAC token for the department ID and the server searches `department_id_index`.
5. **Employee with Highest Salary**: the server orders by `salary_ope DESC` and returns the first encrypted record.
6. **Compare Salaries**: the server retrieves two employees using encrypted full-name tokens and compares their `salary_ope` values.
7. **Employees ordered by Age**: the server sorts records using `age_ope`.
8. **Salary converted to USD**: the server computes $\texttt{Enc(salary)}^{\texttt{rate}}$ using Paillier scalar multiplication. The client decrypts and divides by 100.
9. **Department Payroll Sum**: the server multiplies Paillier salary ciphertexts for all employees in the department, obtaining an encrypted sum.
10. **Oldest Employee**: the server orders by `age_ope DESC` and returns the first encrypted record.
11. **Bonus Calculation**: for eligible employees, the server computes $\texttt{Enc(salary)}^{25}$. The client decrypts and divides by 100 to obtain 25% of salary.

All records returned by the server are verified by the client using HMAC and ECDSA before being decrypted.

6 Security Analysis

6.1 What the Server Does Not Learn

The server does not learn plaintext employee attributes such as names, salaries, ages, emails, phone numbers, job titles, or departments. It also does not learn AES keys, HMAC keys, the Paillier private key, or the ECDSA private key.

Salary values are never stored directly. They are stored as:

- AES ciphertext for retrieval.
- Paillier ciphertext for encrypted arithmetic.
- OPE value for ordering.

6.2 Leakage Discussion

Although plaintext data is protected, the server can still learn some metadata.

Mechanism	**Leakage**
HMAC indexes	Equality patterns. The server can see when two records have the same indexed value.
Repeated queries	The same search value produces the same token, so repeated searches can be detected.
Department search	The server sees how many encrypted records match a department token.
Bonus eligibility search	The server sees how many employees match the eligibility token.
OPE salary	The server learns the relative order of salaries.
OPE age	The server learns the relative order of ages.
Highest salary query	The server sees which encrypted record has the maximum salary order value.
Oldest employee query	The server sees which encrypted record has the maximum age order value.

The most significant leakage comes from OPE because it intentionally preserves order. This is necessary for the server to support ordering and comparison operations directly over encrypted data.

6.3 Leakage Minimization

The system minimizes leakage using the following design decisions:

- Plain attributes are never stored on the server.
- Search indexes are keyed HMACs instead of simple hashes.
- The server does not receive client-side keys.
- Paillier is used for salary sums and scalar multiplication instead of decrypting salaries on the server.
- HMAC and ECDSA allow the client to detect tampering.
- Separate cryptographic representations are only used for fields that need them.

Possible future improvements include query padding, dummy accesses, hiding result sizes, stronger searchable encryption schemes, and replacing the prototype OPE with a more complete Boldyreva OPE implementation.

7 Performance Evaluation

The project includes runtime instrumentation for bootstrap time, encryption time, query latency, and storage overhead. The fixed test client measures the request/response latency for each of the 11 required operations. The evaluation was performed using a dataset with 20 employee records.

7.1 Bootstrap and Encryption Performance

During the bootstrap phase, the client loaded the plaintext CSV dataset, generated or loaded the cryptographic keys, encrypted all employee records, generated the searchable indexes, generated the OPE values, created the Paillier salary ciphertexts, attached HMAC integrity values and ECDSA signatures, and uploaded the encrypted records to the MySQL database.

Metric	Measured Value
Plain records read	20
Encrypted records uploaded	20
Total bootstrap time	651 ms
Total encryption time	368 ms
AES encryption time	25 ms
HMAC index generation time	2 ms
Paillier encryption time	329 ms
OPE encryption time	0 ms
Record HMAC time	1 ms
ECDSA signature time	8 ms

Table 3: Bootstrap and encryption performance.

The results show that Paillier encryption is the most expensive cryptographic operation during bootstrap. This is expected because Paillier uses large modular exponentiations with a 2048-bit key. AES encryption, HMAC index generation, record HMAC computation, and ECDSA signature generation were significantly faster. OPE encryption was negligible for this small dataset.

7.2 Operation Latency

The fixed test client executed all 11 required operations. The measured latencies are shown in Table 4.

Operation	Latency
Search by Employee Identification	76 ms
Search by Employee Full Name	7 ms
Retrieve employees ordered by Salary	38 ms
Search employees by DepartmentID	8 ms
Retrieve employee with highest Salary	3 ms
Compare salaries between two employees	3 ms
Retrieve employees ordered by Age	19 ms
Convert salary to USD	49 ms
Department payroll sum	43 ms
Find oldest employee	4 ms
Compute bonuses for eligible employees	149 ms

Table 4: Latency of supported operations.

The fastest operations were the highest salary query, salary comparison, and oldest employee query. These operations are efficient because they mainly rely on indexed retrieval or ordering over OPE values. The bonus computation was the slowest operation, with a latency of 149 ms, because it required homomorphic scalar multiplication over several Paillier-encrypted salaries. Salary conversion and department payroll sum were also slower than simple searches because they involved Paillier homomorphic operations and client-side decryption.

7.3 Operation Results

The fixed tests also confirmed the correctness of the implemented encrypted operations. Searching by employee identification returned employee `EMP1001`, corresponding to John Smith. Searching by full name returned Emma Johnson. The salary ordering operation returned employees sorted from the lowest salary, Michael Brown with 26,154, to the highest salary, Olivia Thomas with 112,435.

The department query for `DPT109` returned five employees: Emma Johnson, Sophia Taylor, Mia Martin, Ethan Martinez, and Harper Rodriguez. The homomorphic payroll sum for this department produced a decrypted result of 283,075, which corresponds to the sum of the salaries of those five employees.

The salary conversion test used Emma Johnson's salary of 29,794 EUR and a scaled exchange rate of 110, corresponding to an exchange rate of 1.10. The converted salary obtained after homomorphic computation and client-side decryption was 32,773 USD.

The oldest employee query returned Michael Brown, with age 46. The bonus computation returned nine bonus values for employees with bonus eligibility set to `Yes`. These results demonstrate that the server can process the required operations over encrypted data while the client remains responsible for decryption and verification.

7.4 Storage Overhead

Storage overhead was measured by comparing the original plaintext CSV size with the estimated size of the encrypted MySQL table.

Storage Metric	**Value**
Plaintext CSV size	3,486 bytes
Encrypted MySQL table size	180,224 bytes
Storage overhead	51.70×

Table 5: Storage overhead of encrypted database.

The encrypted database is significantly larger than the original plaintext dataset. This overhead is expected because each employee record stores multiple cryptographic representations of some attributes. For example, salary is stored as an AES ciphertext for retrieval, a Paillier ciphertext for homomorphic computation, and an OPE value for ordering. Additionally, searchable HMAC indexes, HMAC integrity values, and ECDSA signatures are stored with each record.

8 Conclusion

This project implements a privacy-preserving employee database using Java and MySQL. The server stores only encrypted employee records and can process the required queries without access to plaintext information or secret keys.

The system combines AES, HMAC-SHA256 indexes, Paillier homomorphic encryption, order-preserving encryption, ECDSA signatures, and HMAC integrity verification. It supports all 11 required operations, including encrypted search, ordering, comparison, salary conversion, department payroll sum, oldest employee retrieval, and bonus calculation.

The main limitations are equality leakage from searchable indexes and order leakage from OPE. These leakages are necessary trade-offs in the implemented design to allow efficient server-side processing over encrypted data.